YAMLess Case Study — getredash/Redash

YAMLess: Generating Azure Infrastructure from Code

Case Study — getredash/Redash

What is YAMLess?

YAMLess is an **Infrastructure from Code (IFC)** tool. Given a code repository, it produces a complete, policy-validated Azure Terraform workspace — with no configuration files, no schema to fill in, and no IaC expertise required from the developer.

The pipeline has four stages:

```
Repo on disk
    │
    ▼
┌─────────────────────┐
│  1. Static Analyzer │  Reads Dockerfile, package.json / requirements.txt /
│                     │  pyproject.toml, .env files, source code.
│                     │  Produces a typed AppProfile with confidence scores.
└──────────┬──────────┘
           │  (fires only when any field is low/medium confidence)
           ▼
┌─────────────────────┐
│  2. LLM Gap-Filler  │  Claude reads relevant source files and fills in what
│                     │  static analysis couldn't determine — via structured
│                     │  tool use only, no free-form parsing.
└──────────┬──────────┘
           │
           ▼
┌─────────────────────┐
│  3. Renderer        │  Jinja2 root templates + static Terraform modules →
│                     │  ready-to-apply workspace. Outputs Container Apps
│                     │  or AKS, selected by --target.
└──────────┬──────────┘
           │
```

```
          ▼
┌──────────────────────┐
│  4. Validator        │  checkov policy-as-code. Security posture is enforced
│                      │  by static module files — the LLM cannot affect it.
└──────────────────────┘
```

Applying YAMLess to getredash/Redash

Redash is a widely-used open-source data visualisation and dashboarding platform. The backend is **Python/Flask** served by Gunicorn, with **rq** workers handling asynchronous query execution. The frontend is a React SPA built separately. Redash uses **Postgres** for metadata, **Redis** as the task broker, and ships as a multi-container application (web + worker + scheduler) from a single repository.

Command run

```
yamless generate /tmp/redash \
  --no-llm \
  --out-dir ~/yamless-outputs/redash-infra \
  --validate
```

Profile detected

Field	Detected value	Confidence	Correct value	Notes
Runtime	`node`	high	`python`	First Dockerfile stage is `FROM node:24` (frontend builder)
Runtime version	`24`	high	`3.13`	Pulled from wrong stage
Port	`5000`	high	`5000`	Correct — `EXPOSE 5000` in runtime stage
Shape	`http-service`	medium	`http-service` + worker	Correct shape; worker process

Field	Detected value	Confidence	Correct value	Notes
				not modelled
Database	`null`	—	`postgres`	Python analyzer never ran; `psycopg2-binary` in pyproject.toml
Cache	`false`	—	`true`	`redis` package in pyproject.toml; not read
Queue/worker	`false`	—	`true`	`rq` package in pyproject.toml; not read
Secrets	`DATABASE_URL`, `REDIS_URL`	high	partial	Correct env vars found; `DATABASE_URL` later dropped (see below)

Output generated

```
redash-infra/
├── main.tf              # Identity, KV, Container App Env, Container App only
├── variables.tf
├── outputs.tf
├── providers.tf
└── modules/
    ├── managed_identity/
    ├── keyvault/
    ├── container_app_env/
    └── container_app/      # Port 5000, http scale trigger, 1 secret only
```

22/22 policy checks passed — but the output is significantly incomplete: no Postgres, no Redis wiring, wrong runtime tag, and `DATABASE_URL` silently absent from Key Vault.

What was correctly captured

- External HTTP ingress on port 5000
- Managed Identity with Key Vault wiring
- Correct scaling profile (HTTP trigger, 1–10 replicas)
- All security golden standards: RBAC-only auth, TLS 1.2, deny-all KV firewall

What was missed — and why

Missing	Root cause
`runtime: python`	Dockerfile stage 1 is `FROM node:24 AS frontend-builder`; analyzer reads the first `FROM` only
Postgres module	Python dependency analyzer never ran because runtime was wrongly set to `node`
Redis wiring	Same — `redis` package in pyproject.toml was never read
rq worker shape	Same — `rq` in pyproject.toml signals background worker; not seen
`DATABASE_URL` in Key Vault	Excluded as “infra-managed” (expected the Postgres module to provision it), but no Postgres module was rendered — it simply disappeared
Worker container profile	Redash runs a separate `rq` worker process; YAMLess has no multi-shape model

Learnings for YAMLess

1. Multi-stage Dockerfile parsing is critical — and cascades into everything else

What happened: Redash’s Dockerfile opens with:

```
FROM node:24-bookworm AS frontend-builder    ← stage 1: build React SPA
...
FROM python:3.13-slim-bookworm               ← stage 2: the actual runtime
```

YAMLess reads only the first `FROM` directive and infers `runtime: node` with high confidence. Because runtime gates which dependency parser runs (`_parse_node()` vs `_parse_python()`), this single mis-read caused a cascading failure: the `pyproject.toml` was never read, so Postgres, Redis, and rq were all invisible to the rest of the pipeline.

Proposed fix: Parse the full Dockerfile stage graph. Identify the terminal stage — the `FROM` that is not referenced as a base by any other stage. Read runtime and version from that stage. This fix also resolves the Ghost runtime-version gap identified in the previous case study.

Impact: Fixing this one bug would have produced the correct profile for Redash with no other changes — Python 3.13 runtime, psycopg2 → Postgres module, redis → cache detected, rq → worker shape hint.

---

2. `DATABASE_URL` silently disappears when no database module is rendered

What happened: `DATABASE_URL` was correctly detected in the env scanner and appeared in `profile.secrets` . However, `_build_context()` treats it as "infra-managed" — meaning it expects the Postgres module to provision the secret. Since no Postgres module was rendered (because database was `null`), `DATABASE_URL` was excluded from `kv_secret_names` and simply never provisioned anywhere.

A developer deploying this output would get a container that fails at startup because `DATABASE_URL` is missing from Key Vault.

Proposed fix: Only treat a secret as infra-managed if its corresponding dependency was actually detected. If `has_postgres = False` but `DATABASE_URL` is in the secrets list, fall through and provision it as a plain Key Vault secret. The logic should be:

```
# Only skip DATABASE_URL if the postgres module will provision it
if s == "DATABASE_URL" and not has_postgres:
    kv_secret_names.append(slug)
elif s not in infra_managed:
    kv_secret_names.append(slug)
```

---

3. Multi-shape applications (web + worker from one repo) need a model

What happened: Redash ships three processes from a single codebase: - `web` — Gunicorn/Flask HTTP server - `worker` — rq worker (background query execution) - `scheduler` — rq-scheduler (periodic tasks)

YAMLess models exactly one shape per profile. The HTTP service shape was correctly inferred (Flask detected), but the worker and scheduler processes have no representation. In production these run as separate container instances with different resource profiles, scale triggers (CPU/queue vs HTTP), and

potentially different images.

Proposed fix: Extend `AppProfile` with an optional `workers` list — a second set of shape + scaling parameters for background processes within the same repo. The renderer would then emit a second Container App (or Deployment + KEDA ScaledObject on AKS) alongside the main web container.

This is the most structurally significant gap identified across all three repos tested so far. Ghost and Outline both embed workers in-process; Redash externalises them, making the gap visible.

---

4. `pyproject.toml` is the primary manifest for modern Python — it must win over `package.json`

What happened: Redash has both a root `package.json` (for the React frontend build tooling) and a root `pyproject.toml` (for the Python backend). When both exist, the current analyzer picks the runtime from the Dockerfile (correct in principle) but then only calls `_parse_node()` because runtime was mis-detected as `node`. Even with a correct runtime fix, the presence of `package.json` at the root should not mislead the analyzer.

Proposed fix: When `pyproject.toml` is present, prefer `_parse_python()` regardless of any Node package manifest — or at minimum, parse both and merge the results, preferring Python signals for backend dependency classification.

Improvement Roadmap (cumulative across all repos)

Priority	Change	Discovered in
High	Multi-stage Dockerfile parser — use terminal stage for runtime	Ghost, Redash
High	Monorepo detection — walk to inner app package.json	Ghost
High	MySQL/MariaDB Terraform module	Ghost

Priority	Change	Discovered in
High	Fix `DATABASE_URL` drop when `has_postgres = False`	Redash
Medium	Multi-shape model — web + worker(s) from one repo	Redash
Medium	Prefer `pyproject.toml` over `package.json` for runtime when both present	Redash
Medium	Recursive secret scanning for monorepos	Ghost
Low	Redis/Azure Cache for Redis Terraform module	Outline, Redash

Summary

Redash is the most revealing test case so far. A single root cause — reading only the first Dockerfile `FROM` stage — cascaded into five separate errors in the profile: wrong runtime, missing database, missing cache, missing worker, and a silently dropped secret. The output technically passed all 22 policy checks, but would fail at deploy time because `DATABASE_URL` is absent from Key Vault.

The multi-stage Dockerfile fix is the highest-leverage improvement in the current backlog: fixing it alone would have produced a near-correct profile for both Ghost and Redash without any other changes.

Generated by YAMLess — https://github.com/zafar-ahmed-kr/YAMLess